

Retrieval-Augmented Generation (RAG)

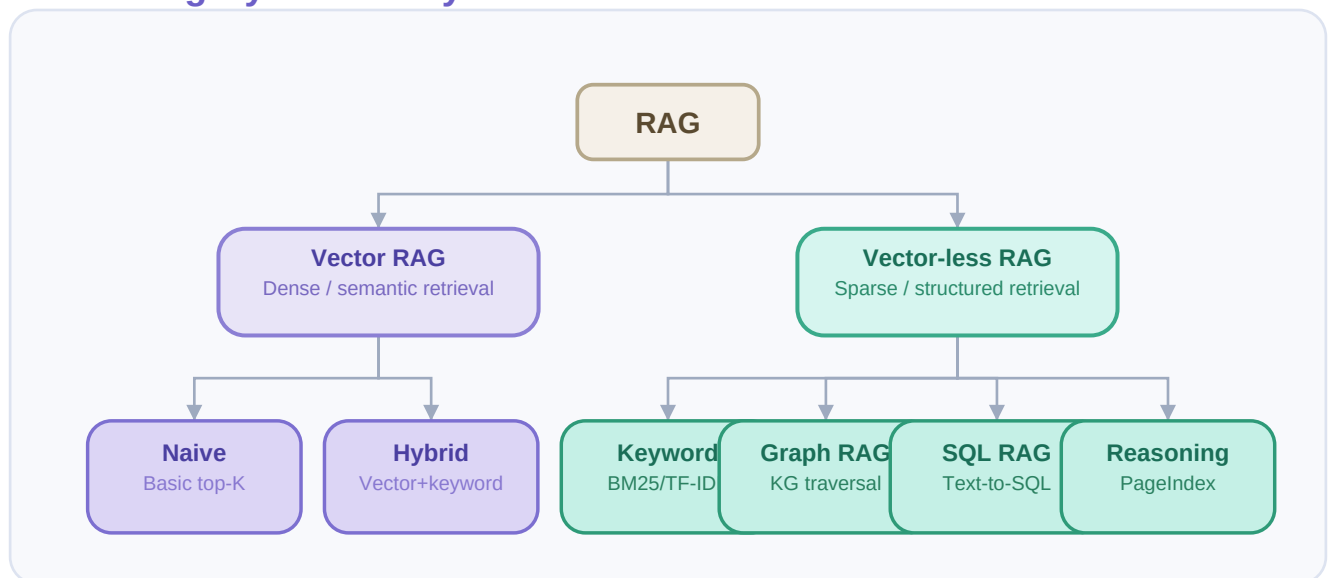
A Comprehensive Guide to RAG Categories, Techniques & Interview Prep

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that supercharges Large Language Model (LLM) responses by dynamically fetching relevant, up-to-date information from an external knowledge source at inference time. Rather than relying purely on the static knowledge baked into model weights during training, a RAG system first **retrieves** the most relevant documents or data for a query, then feeds that context to the LLM to **generate** a grounded and accurate answer.

RAG directly tackles the biggest practical LLM pain-points — hallucination, knowledge staleness, and lack of domain depth — making it the dominant pattern for production AI applications: enterprise search, Q&A bots, legal/financial assistants, code helpers, and analytics agents. RAG approaches split into two high-level families: **Vector RAG** (dense/semantic search using embeddings) and **Vector-less RAG** (sparse, structured, or reasoning-based retrieval without embeddings).

RAG Category Taxonomy



RAG Types — Descriptions

Naive (Vector) RAG

Basic top-K semantic retrieval

Naive RAG is the foundational and most widely deployed RAG pattern. Documents are chunked, embedded via a model such as text-embedding-ada or sentence-transformers, and stored in a vector database (ChromaDB, FAISS, Pinecone). At query time the question is embedded and top-K nearest chunks are retrieved by cosine similarity, then injected into the LLM prompt. Best for general Q&A chatbots, document search, and rapid prototypes where lexical precision is not critical.

Hybrid RAG

Vector search + BM25 keyword retrieval (RRF merge)

Hybrid RAG combines dense vector search with sparse BM25/TF-IDF keyword search, merging ranked results using Reciprocal Rank Fusion (RRF). This dual-retrieval approach captures both semantic meaning and exact keyword matches, boosting recall on technical queries, product codes, or named entities that pure vector search misses. Tools like Elasticsearch or Weaviate natively support hybrid retrieval and are the standard choice for production-grade systems.

Keyword RAG (BM25)

TF-IDF / BM25 — no embeddings required

Keyword RAG uses classical IR algorithms — BM25 or TF-IDF — without any neural embeddings. This makes it extremely fast, zero compute cost at inference, and highly precise for exact-term matching. It excels in domains with unique identifiers such as medical codes, legal citations, part numbers, or financial tickers where semantic similarity is less relevant. Common tools are Elasticsearch and OpenSearch. Low complexity makes it the ideal baseline before adding expensive vector components.

Graph RAG (KG-RAG)

Knowledge graph traversal across entities and relationships

Graph RAG stores knowledge as a property graph (nodes = entities, edges = relationships) and retrieves information via graph traversal rather than similarity search. It is uniquely powerful for multi-hop questions requiring chaining facts across multiple entities — e.g. drug interaction chains, organisational hierarchies, or knowledge graphs over regulatory documents. Built on Neo4j or Amazon Neptune, it is the highest-complexity approach but delivers unmatched reasoning over deeply connected data.

SQL RAG (Text-to-SQL)

LLM-generated SQL queries against live structured databases

SQL RAG bypasses document retrieval entirely: the LLM translates a natural-language question into SQL, executes it against a relational database, and uses the result as context for its final answer. This is ideal for analytics Q&A, business intelligence dashboards, and any scenario where ground truth lives in structured tables. Any SQL database (PostgreSQL, MySQL, BigQuery) combined with LangChain's SQL agent implements this at medium complexity.

Reasoning RAG (PageIndex)

Hierarchical tree index with LLM-driven traversal

Reasoning RAG eschews chunking altogether. Documents are indexed as hierarchical tree structures and the LLM traverses the tree step-by-step to locate relevant pages, reasoning about which branches to explore. This approach handles very long documents — contracts, annual reports, legal briefs — far better than chunk-based methods because it preserves document structure and avoids context fragmentation. Pioneered by VectifyAI's PageIndex; high complexity but state-of-the-art quality on long-form document tasks.

Comparison of RAG Techniques

Approach	Retrieval Engine	Best For	Key Tools	Complexity
Naive Vector RAG	Single embedding model, cosine similarity	General text, Q&A; bots	ChromaDB, FAISS	Low
Hybrid RAG	Vector search + BM25, RRF merge	Production systems need both	Elasticsearch + any VDB	Medium
Keyword RAG (BM25)	TF-IDF / BM25, no embeddings	Codes, jargon, IDs, citations	Elasticsearch, OpenSearch	Low
Graph RAG (KG-RAG)	Knowledge graph traversal	Relationships, multi-hop Q&A;	Neo4j, Amazon Neptune	High
SQL RAG (Text-SQL)	LLM generates SQL, queries live DB	Structured data, analytics Q&A;	Any SQL DB + LangChain	Medium
Reasoning RAG (PageIndex)	Hierarchical tree index + LLM traversal	Long docs, finance, legal, no chunking	PageIndex (VectifyAI)	High

Top 10 RAG Interview Questions & Answers

1 What is RAG and why is it preferred over fine-tuning?

RAG augments LLM responses by retrieving external knowledge at inference time, so the model always has access to current, domain-specific information without retraining. Fine-tuning bakes knowledge into weights — expensive, slow to update, and prone to catastrophic forgetting. RAG separates knowledge storage (the retrieval index) from reasoning (the LLM), making it cheaper, updatable in real-time, and auditable via retrieved sources.

2 Explain the difference between Naive RAG and Hybrid RAG.

Naive RAG uses only dense vector search (embedding similarity) to retrieve chunks. Hybrid RAG combines dense vector search with sparse BM25 keyword search, merging results via Reciprocal Rank Fusion (RRF). Hybrid RAG outperforms Naive RAG on queries with specific terms, product codes, or named entities that vector search can miss due to out-of-vocabulary or low-frequency issues.

3 What is chunking and why does chunk size matter in RAG?

Chunking is splitting documents into smaller pieces before embedding them. Chunk size is critical: too small and individual chunks lack context; too large and they dilute relevance and exceed LLM context windows. Common strategies include fixed-size chunking (e.g. 512 tokens with overlap), sentence-based chunking, paragraph-based chunking, and hierarchical chunking. The best size depends on document structure and the embedding model's optimal input length.

4 What is Reciprocal Rank Fusion (RRF) and when is it used?

RRF is a rank aggregation algorithm that merges ranked lists from multiple retrieval sources without needing score normalisation. Each document's fused score is computed as the sum of $1/(\text{rank} + k)$ across all lists (k is typically 60). It is used in Hybrid RAG to combine vector search rankings with BM25 rankings into a single re-ranked list, consistently outperforming individual retrievers and requiring no tuning of score weighting.

5 What is the difference between Vector RAG and Graph RAG?

Vector RAG retrieves semantically similar text chunks using embedding-based similarity — fast and general-purpose but treats each chunk independently. Graph RAG represents knowledge as a property graph and retrieves via traversal, enabling multi-hop reasoning across entity relationships. Graph RAG excels at complex relational queries ('Who collaborated with X on Y while working at Z?') that are impossible to answer from individual isolated text chunks.

6 How does SQL RAG (Text-to-SQL) work, and what are its limitations?

SQL RAG prompts the LLM to translate a natural-language query into SQL, executes it against a live database, and feeds the result back to the LLM for answer generation. Limitations include: schema complexity (large schemas confuse the model), SQL dialect differences, ambiguous column naming, risk of incorrect queries returning wrong data silently, and inability to answer questions requiring unstructured reasoning that SQL cannot express.

7 What evaluation metrics are used to assess RAG system quality?

Key metrics include: Context Precision (are retrieved chunks relevant?), Context Recall (are all necessary facts retrieved?), Faithfulness (does the answer only use retrieved context, no hallucination?), and Answer Relevance (does the answer address the query?). Frameworks like RAGAS automate these using LLM-as-judge evaluation. Retrieval-level metrics (MRR, NDCG, Hit@K) measure retriever quality independently of the generator.

8 What is the role of re-ranking in RAG pipelines?

Initial retrieval returns top-K candidates quickly but imprecisely. A re-ranker (cross-encoder model or LLM-as-judge) then re-scores each candidate against the query with deeper computation, promoting the most relevant chunks before passing them to the LLM. Re-ranking dramatically improves faithfulness and reduces noise in the context window. Popular re-rankers include Cohere Rerank, BGE-Reranker, and FlashRank.

9 What is semantic chunking and how does it differ from fixed-size chunking?

Semantic chunking splits documents at natural semantic boundaries — where topic or meaning shifts — rather than at fixed token counts. It uses embedding similarity between consecutive sentences to detect boundaries: if adjacent sentences diverge significantly in embedding space, a chunk boundary is inserted. This produces coherent, self-contained chunks that improve retrieval precision compared to fixed-size chunks that can split sentences or mix unrelated topics.

10 What are the main failure modes of RAG systems and how do you mitigate them?

Key failure modes: (1) Retrieval failure — wrong chunks retrieved; mitigate with Hybrid RAG + re-ranking + query rewriting. (2) Context window overflow — too many chunks exceed LLM limits; mitigate with tighter top-K and re-ranking. (3) Hallucination despite retrieval — LLM ignores context; mitigate with faithfulness-focused prompting and citation checks. (4) Stale index — outdated documents; mitigate with incremental indexing pipelines. (5) Prompt injection via retrieved content; mitigate with input sanitisation and guardrails.